Cyberscope

A TAC Security Company

Audit Report

EZManager

January 2026

Repository https://github.com/Tylergcantrell/EzManager

Commit c7f86daa7e02bea10476f90dc86b15cc9707da41

Audited by © cyberscope

Table of Contents

Table of Contents 1
Risk Classification 3
Review 4
Audit Updates 4
Source Files 4
Overview 5
CLManager 5
UniswapAdapter 6
AerodromeAdapter 6
Findings Breakdown 8
Diagnostics 9
EBF - Excessive Bot Fee 11
Description 11
Recommendation 12
UQA - Unconstrained Quoted Amount 13
Description 13
Recommendation 14
DIZP - Decimal Induced Zero Price 15
Description 15
Recommendation 16
PTAI - Potential Transfer Amount Inconsistency 17
Description 17
Recommendation 17
CSA - Compounded Slippage Application 18
Description 18
Recommendation 18
CVD - Core Value Discrepancy 19
Description 19
Recommendation 20
ITV - Incomplete Tick Validation 21
Description 21
Recommendation 21
IUV - Incomplete USDC Validation 23
Description 23
Recommendation 24
IPS - Inconsistent Pairing Semantics 25
Description 25
Recommendation 26
MPV - Missing Pool Validation 27

Description 27
Recommendation 28
PUPT - Potentially Unsorted Pair Tokens 29
Description 29
Recommendation 30
RTT - Redundant Token Transfers 31
Description 31
Recommendation 33
UPI - Uninitialized Pool Inconsistency 34
Description 34
Recommendation 36
UCS - Unreachable Code Segment 37
Description 37
Recommendation 38
Summary 39
Disclaimer 40
About Cyberscope 41

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation**: This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation**: This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical**: Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium**: Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor**: Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative**: Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
● Critical	Highly Likely / High Impact
● Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
● Minor / Informative	Unlikely / Low to no Impact

Review

Repository	https://github.com/Tylergcantrell/EzManager
Commit	c7f86daa7e02bea10476f90dc86b15cc9707da41

Audit Updates

Initial Audit	11 Jan 2026

Source Files

Filename	SHA256
AerodromeAdapter.sol	70a040d411b07d1d67be3b36b8c4c056500412f82b64311138c1848ac1b74267
CLManager.sol	6643aa1ec88ad5c043e48d5dff947441eb641b71d45ed27d4ff8043ce3029777
UniswapAdapter.sol	8e6d2a16f04a8b0c59fd06ed4f9acf615e2556232e30b9ccadb5bb32a8dcf296

Overview

The system implements a Concentrated Liquidity Management System with three adapter contracts: CLManager orchestrates position lifecycle operations, while UniswapAdapter and AerodromeAdapter provide DEX-specific implementations for Uniswap V3/PancakeSwap V3 and Aerodrome Slipstream respectively. The architecture separates business logic (CLManager) from DEX integration (adapters), enabling modular support for multiple concentrated liquidity protocols.

CLManager

CLManager serves as the user-facing orchestration layer for all position lifecycle operations. It is an upgradeable, owner-controlled contract responsible for coordinating between CLCore (state), Valuation (pricing), RebalancePlanner (optimization), and DEX adapters (execution).

Key Responsibilities:

Position Lifecycle: Manages opening, closing, and modifying positions through coordinated calls to adapters and core state updates. Handles collateral additions/removals, fee collection/compounding, and range changes.

Fee Management: Deducts protocol fees (default 40 BPS) on deposits and sends them to ProtocolReserve. Supports bot automation with configurable bot fees (default 20 BPS) for exitPosition, collectFeesToUSDC, compoundFees, and changeRange operations.

Dust Accounting: Tracks leftover USDC from mint/add operations via _addDust(), pushing funds to CLCore and updating per-position dust balances. Provides withdrawDust for users to reclaim tracked dust.

Pair Validation: Enforces supported pair restrictions (USDC/token, USDC/WETH, WETH/token) using TWAP-based valuation checks to prevent malicious or unsupported token pairs.

UniswapAdapter

UniswapAdapter is a DEX adapter contract implementing the ICLDexAdapter interface for Uniswap V3 and PancakeSwap V3 pools. It translates high-level manager operations into Uniswap V3 NonfungiblePositionManager (NPM) calls.

Key Responsibilities:

Position Minting: Seeds token pairs (direct USDC or via WETH bridge), executes rebalance swaps, and calls NPM.mint() to create LP NFT positions. Returns dust to manager as leftover USDC.

Liquidity Management: Implements addLiquidity (increaseLiquidity) and removeLiquidityBpsUSDC (decreaseLiquidity + collect + swap to USDC) with automatic token sorting and slippage protection.

Fee Collection: Collects fees via NPM.collect() with amount0Max/amount1Max set to type(uint128).max, ensuring all tokensOwed are retrieved. Swaps collected fees to USDC for standardized accounting.

Position Unwinding: Fully exits positions via decreaseLiquidity (burn all liquidity) + collect, returning tokens to specified receiver. Used by changeRange and exitPosition flows.

AerodromeAdapter

AerodromeAdapter is a DEX adapter contract implementing the ICLDexAdapter interface for Aerodrome Slipstream pools. It provides identical functionality to UniswapAdapter but handles Aerodrome's tickSpacing-based pool identification and interface variations.

Key Responsibilities:

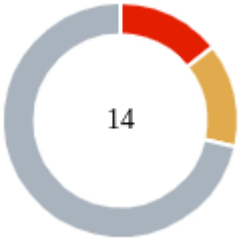
Position Minting: Seeds token pairs, executes rebalance swaps, and calls NPM.mint() with Aerodrome-specific MintParams (includes sqrtPriceX96 parameter). Returns dust to the manager as leftover USDC.

Liquidity Management: Implements addLiquidity (increaseLiquidity) and removeLiquidityBpsUSDC (decreaseLiquidity + collect + swap to USDC) with tickSpacing-based pool identification.

Fee Collection: Collects fees via NPM.collect() with amount0Max/amount1Max set to type(uint128).max, ensuring all tokensOwed are retrieved. Swaps collected fees to USDC for standardized accounting.

Position Unwinding: Fully exits positions via decreaseLiquidity (burn all liquidity) + collect, returning tokens to specified receiver. Used by changeRange and exitPosition flows.

Findings Breakdown



- Critical 2
- Medium 2
- Minor / Informative 10

Severity	Unresolved	Acknowledged	Resolved	Other
Critical	2	0	0	0
Medium	2	0	0	0
Minor / Informative	10	0	0	0

Diagnostics

Critical Medium Minor / Informative

Severity	Code	Description	Status
Critical	EBF	Excessive Bot Fee	Unresolved
Critical	UQA	Unconstrained Quoted Amount	Unresolved
Medium	DIZP	Decimal Induced Zero Price	Unresolved
Medium	PTAI	Potential Transfer Amount Inconsistency	Unresolved
Minor / Informative	CSA	Compounded Slippage Application	Unresolved
Minor / Informative	CVD	Core Value Discrepancy	Unresolved
Minor / Informative	ITV	Incomplete Tick Validation	Unresolved
Minor / Informative	IUV	Incomplete USDC Validation	Unresolved
Minor / Informative	IPS	Inconsistent Pairing Semantics	Unresolved
Minor / Informative	MPV	Missing Pool Validation	Unresolved
Minor / Informative	PUPT	Potentially Unsorted Pair Tokens	Unresolved
Minor / Informative	RTT	Redundant Token Transfers	Unresolved
Minor / Informative	UPI	Uninitialized Pool Inconsistency	Unresolved

●	UCS	Unreachable Code Segment	Unresolved

EBF - Excessive Bot Fee

Criticality	Critical
Location	CLManager.sol#L666,990
Status	Unresolved

Description

In the `compoundFees` function, the bot fee is intended to be a percentage of the newly compounded trading fees. However, the current implementation removes a proportional share of the entire position, comprising both the user's original principal and the compounded fees, to pay the bot. This results in the bot receiving compensation partially derived from the user's principal, which violates the intended fee model.

The function follows these steps:

1. Collects accrued trading fees from the position`.
2. Rebalances and adds these fees back into the position`.
3. Calculates the USDC value of the compounded fees.
4. Computes the target bot fee as a percentage of the compounded fees.
5. Retrieves the total position value after compounding.
6. Calculates the fraction of the position to remove.
7. Removes this fraction from the entire position.

In steps 6 and 7, the source of the funds is the entire position which may include pre-existing principal amounts. The bot receives a proportional slice of the entire position, meaning the fee is funded not just from the compounded trading fees but also from the user's original deposit.

```shell
uint256 botFeeUSDC =
ICLDexAdapter(ctx.dex).removeLiquidityBpsUSDC(
         abi.encode(ctx.tokenId),
         feeFraction,
         address(USDC),
         address(this),
         slippageBps

         );
```

Recommendation

To align with the intended fee model, the bot fee should be derived solely from the compounded trading fees. This ensures that the user's original principal remains untouched, preserving capital integrity and maintaining trust in the fee mechanism.

UQA - Unconstrained Quoted Amount

Criticality	Critical
Location	UniswapAdapter.sol#L746 AerodromeAdapter.sol#L748
Status	Unresolved

Description

The contract uses the `quoteExactInputSingle` function from the `IUniswapQuoterV2Like` interface to estimate output amounts for token swaps. This function is invoked in both `_swapExactInSingleHop` and `_quoteExactInSingleHop` to obtain price quotes. However, in both cases, the `sqrtPriceLimitX96` parameter is explicitly set to `0`, which instructs the Uniswap V3 Quoter to use the current spot price without any time-weighted average price (TWAP) validation.

This means the contract relies solely on the instantaneous spot price for quoting, which may be susceptible to short-term price manipulation or volatility, especially in low-liquidity pools and flash-loan scenarios.

```shell
IUniswapQuoterV2Like.QuoteExactInputSingleParams
memory qp =

IUniswapQuoterV2Like.QuoteExactInputSingleParams({
        tokenIn: tokenIn,
        tokenOut: tokenOut,
        amountIn: amountIn,
        fee: fee,
        sqrtPriceLimitX96: 0
    });

try
IUniswapQuoterV2Like(uniswapQuoter).quoteExactInpu
tSingle(qp)
```

```
    returns (uint256 quoted, uint160, uint32,
uint256)
{
    if (quoted != 0) {
        minOut = FullMath.mulDiv(quoted, (BPS -
sl), BPS);
    }
} catch {
    revert QuoterFailed();

}
```

Recommendation

Consider using a time-weighted average price (TWAP) to constrain the price range used for quoting. This approach reduces the risk of relying on potentially manipulated spot prices and enhances the reliability of swap estimates in volatile or low-liquidity markets.

DIZP - Decimal Induced Zero Price

Criticality	Medium
Location	CLManager.sol#L1258 Valuation.sol#L523
Status	Unresolved

Description

The `CLManager` contract relies on the `VALUATION.usdcValue(dex, token, amount)` function to estimate the USDC value of a given token. This function internally calls `_usdcValueAerodrome` or `_usdcValueUniswap`, which in turn use `_spotQuote` to compute the price based on Uniswap V3-style TWAPs.

The `_spotQuote` function calculates the output amount using the formula:

```shell
FullMath.mulDiv(amountIn, Q192, priceX192)
```

This formula assumes that `priceX192` is scaled to 2^192. However, if the price value is derived from a token with more than 18 decimals, the result of this multiplication and division may result zero due to the mismatch in decimal precision. As a result, the `usdcValue` function may return zero for tokens with high decimal counts.

This behavior can cause discrepancies in the `CLManager` contract. For example, during the `_ensureSupportedPair` check, if the USDC value of a token is incorrectly returned as zero, the contract will reject the token pair as unsupported. Additionally, if a zero valuation passes through to minting logic, it may cause a revert due to invalid or zero amounts being used.

Recommendation

To ensure accurate valuations across a broader range of tokens, the valuation logic should account for differences in token decimal precision. This adjustment will prevent zero-value results when handling tokens with more than 18 decimals, ensuring consistent behavior in token pair validation and minting operations.

PTAI - Potential Transfer Amount Inconsistency

Criticality	Medium
Location	UniswapAdapter.sol#L437 AerodromeAdapter.sol#L383
Status	Unresolved

Description

The `transfer()` and `transferFrom()` functions are used to transfer a specified amount of tokens to an address. The fee or tax is an amount that is charged to the sender of an ERC20 token when tokens are transferred to another address. According to the specification, the transferred amount could potentially be less than the expected amount. This may produce inconsistency between the expected and the actual behavior. As a result requirements for exact token balance in the adapter may fail if fees are applied.

```shell
_requireBalance(token0, amount0);

_requireBalance(token1, amount1);
```

Recommendation

The team is advised to take into consideration the actual amount that has been transferred instead of the expected. It is important to note that an ERC20 transfer tax is not a standard feature of the ERC20 specification, and it is not universally implemented by all ERC20 contracts. Therefore, the contract could produce the actual amount by calculating the difference between the transfer call.

CSA - Compounded Slippage Application

Criticality	Minor / Informative
Location	CLManager.sol
Status	Unresolved

Description

In the `CLManager` contract, a single slippage value (`slippageBps`) is reused across multiple sequential operations, such as token swaps and liquidity additions. Each operation applies the slippage constraint to the result of the previous one, rather than to the original input amount. This cumulative application can lead to a compounded slippage effect, where the final deviation from the initial amount exceeds the user's intended tolerance. As a result, users may experience greater value loss than expected during complex transactions involving multiple steps.

Recommendation

The team is advised to apply a consistent slippage amount across all operations based on the original input amount. This approach ensures that the slippage tolerance remains within the user's expectations and prevents unintended value loss due to compounding effects.

CVD - Core Value Discrepancy

Criticality	Minor / Informative
Location	CLManager.sol#L611,672,995
Status	Unresolved

Description

The Manager contract performs operations on the CORE contract's position, including collecting fees from the position's active liquidity. The methods `changeRange`, `compoundFees`, and `collectFeesToUSDC` accrue a BOT fee, which is deducted from the active liquidity of the position. However, this deduction modifies internal metrics within the CORE contract—such as the `totalDepositedUSDC` value—without updating them accordingly. Since `totalDepositedUSDC` is typically updated during liquidity-altering operations, this inconsistency can lead to a mismatch in the contract's state, increasing the risk of errors and unreliable data.

```shell
function changeRange(
    bytes32 key,
    int24 newTickLower,
    int24 newTickUpper,
    uint256 slippageBps
)
    external
    nonReentrant
    onlyKeyOwnerOrBot(key)
    whenNotPaused
{ ...
 ICLDexAdapter(p.dex).unwindToTokens(abi.encode(p.tokenId),
address(this));
... }
```

Recommendation

The team is advised to review the implementation and ensure that the CORE metrics are synchronized and consistently reflect the current state of the position at all times. This will improve data consistency and enhance the reliability of the system's behavior.

ITV - Incomplete Tick Validation

Criticality	Minor / Informative
Location	CLManager.sol#L443
Status	Unresolved

Description

The contract performs validation to ensure that the tick values used in liquidity positions align with the required tick spacing. However, it does not verify whether the tick values fall within the valid range defined by the Uniswap V3 protocol, which is between -887272 and 887272. This omission can lead to the use of invalid tick values, potentially causing unexpected behavior or failed transactions when interacting with Uniswap V3 pools.

```shell
if ((tickLower % tickSpacing) != 0 || (tickUpper %
tickSpacing) != 0) revert TickAlignmentError();

if (tickLower >= tickUpper) revert InvalidTickRange();
```

Recommendation

To ensure tick values are both aligned and within the acceptable bounds, the contract should include an additional check:

```shell
if (arithmeticMeanTick < -887272 ||
arithmeticMeanTick > 887272) {
    revert InvalidTickRange();

}
```

This validation would prevent the use of out-of-range tick values, ensuring compatibility with Uniswap V3 and maintaining the integrity of position management.

IUV - Incomplete USDC Validation

Criticality	Minor / Informative
Location	CLManager.sol#L1258
Status	Unresolved

Description

The `_seedPair` function includes a conditional check that prevents the processing of WETH-WETH trading pairs by reverting the transaction when both `token0` and `token1` are WETH. However, it does not apply the same restriction to USDC-USDC pairs, allowing them to proceed without issue. This inconsistency in validation logic could lead to unexpected behavior or misconfigurations.

```shell
function _seedPair(
        address dex,
        address token0,
        address token1,
        uint256 netUSDC,
        uint256 slippageBps
    ) internal returns (uint256 amount0In, uint256
amount1In) {
        bool token0IsUSDC = token0 ==
address(USDC);
        bool token1IsUSDC = token1 ==
address(USDC);
        bool token0IsWETH = token0 ==
address(WETH);
        bool token1IsWETH = token1 ==
address(WETH);

        if (token0IsUSDC || token1IsUSDC) {
            USDC.safeTransfer(dex, netUSDC);
            amount0In = token0IsUSDC ? netUSDC :
```

```
0;
            amount1In = token1IsUSDC ? netUSDC :
0;
        }
        ...

}
```

Recommendation

To maintain consistent validation logic and prevent potential misconfigurations, consider applying a similar restriction to USDC-USDC pairs as is done for WETH-WETH pairs. This ensures uniform handling of identical-token pairs and reduces the risk of unintended behavior.

IPS - Inconsistent Pairing Semantics

Criticality	Minor / Informative
Location	UniswapAdapter.sol#L361
Status	Unresolved

Description

The `_pairingForPool` function contains a logical inconsistency in how it handles token pair combinations involving WETH. Specifically, the function first checks whether either `token0` or `token1` is equal to `WETH`:

```shell
bool token0IsWeth = token0 == WETH;
bool token1IsWeth = token1 == WETH;

if (!(token0IsWeth || token1IsWeth)) revert
BadParam();
```

This condition ensures that at least one of the tokens is WETH. However, immediately afterward, the function checks whether both tokens are WETH and reverts if so:

```shell
address other = token0IsWeth ? token1 : token0;

if (other == WETH) revert BadParam();
```

This logic is redundant and potentially misleading. If both `token0` and `token1` are WETH, the function would have already returned earlier in the block that handles the `token0 == token1` case:

```shell
if (token0 == token1) {
    if (token0 == WETH) return
PairingType.WETH_BRIDGED;
    ...

}
```

As a result, the later check for both tokens being WETH is unreachable and potentially highlights an inconsistency in the intended functionality. Currently if both tokens are `WETH` the method always returns `WETH_BRIDGED` and does not revert. The case is similar when both tokens are `USDC`.

Recommendation

The team is advised to review the implementation to ensure it aligns with the intended design. Eliminating inconsistencies will promote system integrity and enhance user trust.

MPV - Missing Pool Validation

Criticality	Minor / Informative
Location	CLManager.sol#L423
Status	Unresolved

Description

In the `openPosition` function of the `CLManager` contract, when interacting with Aerodrome pools, the implementation does not validate that the provided pool address matches the expected pool derived from the factory, token pair, and tick spacing. This is inconsistent with the Uniswap path, where the contract explicitly verifies the pool address against the factory's derived address for the given tokens and fee tier, reverting if there is a mismatch.

```shell
if (isAero) {
    tickSpacing = IUniswapV3Pool(pool).tickSpacing();
    if (tickSpacing <= 0) revert InvalidTickSpacing();
    uint24 aeroFee =
IAerodromeFactory(factory).tickSpacingToFee(tickSpacing);
    if (aeroFee == 0) revert AeroFeeZero();
    fee = aeroFee;
    poolParam = uint24(uint256(int256(tickSpacing)));
} else {
    fee = IUniswapV3Pool(pool).fee();
    if (fee == 0) revert InvalidParams();
    tickSpacing = IUniswapV3Pool(pool).tickSpacing();
    if (tickSpacing <= 0) revert InvalidTickSpacing();
    address derivedPool =
IUniswapV3Factory(factory).getPool(token0, token1, fee);
    if (derivedPool != pool) revert InvalidParams();
    poolParam = fee;}
```

Recommendation

To ensure consistency and prevent potential misconfigurations, it is advised to validate the Aerodrome pool address against the expected address derived from the factory, token pair, and tick spacing. This validation helps maintain integrity and reduces the risk of interacting with incorrect or malicious pool contracts.

PUPT - Potentially Unsorted Pair Tokens

Criticality	Minor / Informative
Location	RebalancePlanner.sol#L128
Status	Unresolved

Description

The `_poolAddress` function internally sorts the token pair to ensure a consistent ordering when querying the pool address. This sorting aligns with how Uniswap V3 and similar protocols define token0 and token1 within a pool. However, in the `_solveIterative` function, the input amounts `amount0` and `amount1` are used without applying the same sorting logic.

As a result, there is a potential mismatch between the token ordering assumed by the pool and the ordering of the input amounts. This discrepancy can lead to incorrect calculations, particularly when deriving price or value metrics based on the pool's `sqrtPriceX96`, which reflects the price of the sorted token pair. For example, the algorithm may interpret `amount0` as the quantity of token0, when in fact it corresponds to token1 in the sorted pool context. This can potentially lead to suboptimal or incorrect swap planning outcomes.

The case is similar with the adapters, which extensively rely on the manager providing sorted pairs. A more defensive approach is advisable.

```shell
function _poolAddress(bool isSlip, address factory, address
t0, address t1, uint24 param) internal view returns
(address) {
if (t0 == t1) return address(0);
(address tokenA, address tokenB) = t0 < t1 ? (t0, t1) :
(t1, t0);

if (isSlip) {
// param is treated as tickSpacing for Aerodrome
int24 ts = int24(int256(uint256(param)));
try IAerodromeFactory(factory).getPool(tokenA, tokenB, ts)
returns (address p) {
if (_poolAllowed(p)) return p;
} catch {}
} else {
// param is fee for UniV3
try IUniswapV3Factory(factory).getPool(tokenA, tokenB,
param) returns (address p) {
if (_poolAllowed(p)) return p;
} catch {}
}
return address(0);

}
```

Recommendation

Current invocations of the `planFromTokenBundle` function retrieve the `token0` and `token1` addresses directly from the pool. However, the team is advised to ensure consistent handling of token addresses within the planner itself. This approach can help prevent future inconsistencies, while also improving the upgradeability and maintainability of the system.

RTT - Redundant Token Transfers

Criticality	Minor / Informative
Location	CLManager.sol#L1196
Status	Unresolved

Description

The `_seedPair` function performs redundant token transfers between the CLManager and the adapter. Specifically, when seeding a position with USDC or WETH, the function transfers tokens to the adapter, which are then routed back to the CLManager, only to be sent again to the adapter. This architecture introduces unnecessary token transfers, increases gas costs and adds complexity.

Shell

```
function _seedPair(
    address dex,
    address token0,
    address token1,
    uint256 netUSDC,
    uint256 slippageBps
) internal returns (uint256 amount0In, uint256
amount1In) {
    bool token0IsUSDC = token0 == address(USDC);
    bool token1IsUSDC = token1 == address(USDC);
    bool token0IsWETH = token0 == address(WETH);
    bool token1IsWETH = token1 == address(WETH);

    if (token0IsUSDC || token1IsUSDC) {
        USDC.safeTransfer(dex, netUSDC);
        amount0In = token0IsUSDC ? netUSDC : 0;
        amount1In = token1IsUSDC ? netUSDC : 0;
    } else {
        if (token0IsWETH == token1IsWETH) {
            revert UnsupportedPair();
        }
        USDC.safeTransfer(dex, netUSDC);
        uint256 wethSeed =
ICLDexAdapter(dex).swapExactUSDCForWETH(
            address(USDC),
            netUSDC,
            slippageBps,
            address(this)
        );
        if (wethSeed > 0) {
            WETH.safeTransfer(dex, wethSeed);
        }
        amount0In = token0IsWETH ? wethSeed : 0;
        amount1In = token1IsWETH ? wethSeed : 0;
    }

}
```

Recommendation

By simplifying the token flow to avoid unnecessary transfers between the CLManager and the adapter, the function can reduce gas costs and simplify execution logic.

UPI - Uninitialized Pool Inconsistency

Criticality	Minor / Informative
Location	CLManager.sol UniswapAdapter.sol AerodromeAdapter.sol
Status	Unresolved

Description

In the adapter contracts, the `_computeSlippageMins` function checks whether a pool is uninitialized by verifying if `sqrtPriceX96 == 0`. If so, it returns fallback slippage bounds based on simple percentage reductions of token balances. This fallback does not halt execution or raise an error, allowing the process to continue with seemingly valid minimums. However, when the adapter subsequently attempts to mint or increase liquidity via Uniswap V3's `NonfungiblePositionManager`, the operation will revert because the pool is not yet initialized—Uniswap requires a non-zero `sqrtPriceX96` to accept liquidity. This results in a low-level revert with a non-descriptive error, making it difficult for users to understand the cause.

Furthermore, the `_ensureSupportedPair` function in the `CLManager` contract performs a check to validate token pairs by ensuring both tokens have non-zero USDC values. While this approach works for direct USDC pairs, it fails to detect uninitialized pools in WETH-bridged pairs. For example, if the WETH/USDC and TOKEN/USDC pools are initialized but the TOKEN/WETH pool is not, the valuation check may still pass due to successful routing through alternative paths. As a result, the system may proceed to open a position in an uninitialized pool, only to fail later during the minting process.

Shell

```
function _ensureSupportedPair(address dex, address token0,
address token1) internal view {
    if (token0 == address(0) || token1 == address(0))
revert UnsupportedPair();

    bool token0IsUSDC = token0 == address(USDC);
    bool token1IsUSDC = token1 == address(USDC);
    if (token0IsUSDC || token1IsUSDC) {
        // still perform valuation check below
    }

    bool token0IsWETH = token0 == address(WETH);
    bool token1IsWETH = token1 == address(WETH);
    if (!token0IsUSDC && !token1IsUSDC) {
        if (!((token0IsWETH || token1IsWETH) &&
!(token0IsWETH && token1IsWETH))) revert UnsupportedPair();
    }

    // Valuation smoke-test: ensure both tokens have a
non-zero USDC value through the VALUATION oracle.
    // Use a safe test amount that works across typical
ERC20 decimals: 1e18 (WAD).
    uint256 testAmount = 1e18;
    // token0
    uint256 v0 = VALUATION.usdcValue(dex, token0,
testAmount);
    if (v0 == 0) revert UnsupportedPair();

    // token1
    uint256 v1 = VALUATION.usdcValue(dex, token1,
testAmount);
    if (v1 == 0) revert UnsupportedPair();

}
```

Recommendation

By explicitly checking for pool initialization (e.g., verifying that `sqrtPriceX96` is non-zero) before proceeding with liquidity operations, the system can catch uninitialized pools early. This prevents downstream reverts and provides clearer, more actionable error messages to users.

UCS - Unreachable Code Segment

Criticality	Minor / Informative
Location	UniswapAdapter.sol#L361
Status	Unresolved

Description

The `_pairingForPool` function contains a conditional block where the final `if (hasDirect)` statement is never executed due to the logic of the preceding conditions. Specifically, the earlier condition:

```shell
if (hasDirect && (!hasBridge || directLiq >= bridgeLiq))
return PairingType.USDC_DIRECT;
```

already covers all scenarios in which `hasDirect` is true. If `hasDirect` is true and either `hasBridge` is false or `directLiq` is greater than or equal to `bridgeLiq`, the function returns early. If `hasBridge` is true and the previous condition fails, the next condition:

```shell
if (hasBridge) return PairingType.WETH_BRIDGED;
```

will be true, and the function will return. Therefore, by the time execution reaches:

```shell
if (hasDirect) return PairingType.USDC_DIRECT;
```

either the function has already returned or `hasDirect` is false, making this line unreachable.

This unreachable code may indicate a logical oversight or redundant condition that should be removed to improve code clarity and maintainability.

Recommendation

The team is advised to review the implementation to ensure it aligns with the intended design. Eliminating inconsistencies will promote system integrity and enhance user trust.

Summary

The EZManager protocol proposes a liquidity management system, providing modular tracking, accounting, and valuation of LP positions across Uniswap V3 and Aerodrome Slipstream. This audit investigates security issues, business logic concerns and potential improvements.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.

A **TAC Security** Company

The Cyberscope team

cyberscope.io